

1. a. Secret Key Cryptography:

Maintaining message integrity over their vulnerable communication channel, Alice and Bob employ secret key cryptography. Alice encrypts her messages using a shared secret key and computes a Message Authentication Code (MAC) via a cryptographic hash function, combining it with the encrypted message. Upon receiving the message, Bob decrypts it using the same secret key and recalculates the MAC. By comparing the computed MAC with the received one, Bob verifies the message's integrity, detecting any unauthorized modifications that might have occurred during transmission.

b. Cryptographic Hash Function:

In addition to secret key cryptography, Alice and Bob utilize cryptographic hash functions to ensure message integrity. Alice appends a hash of the message, generated using a secret key known only to both parties, to the message itself before transmission. Upon receipt, Bob independently computes the hash of the received message using the shared secret key. By comparing the computed hash with the received one, Bob can confirm the integrity of the message, detecting any alterations that might have been made en route.

c. Public Key Cryptography:

Furthermore, Alice and Bob leverage public key cryptography to guarantee the integrity of their messages. Alice signs her messages using her private key, creating a digital signature. Bob, possessing Alice's public key obtained from her certificate, verifies the signature upon receiving the message. The verification process ensures that the message originated from Alice and remains unaltered during transmission. This technique allows both parties to detect any unauthorized modifications or forgeries while communicating over their insecure channel.

2. Appending a hash of each message doesn't entirely solve the problem of preventing message modification by an attacker. While this approach can detect unintentional changes to the message during transmission, it fails to address the vulnerability of the hash itself being intercepted and modified along with the message. If an attacker can modify the message, they can also compute a new hash for the modified message and replace the original hash. When the recipient recalculates the hash of the received message and compares it to the received hash, they'll find a match, unaware of the message's tampering. As a result, simply appending a hash of the message doesn't provide a robust defense against a determined attacker who can modify both the message and its corresponding hash.
3. Hash functions possess several crucial properties to ensure their effectiveness and security in various applications. Firstly, **Collision Resistance** is vital, ensuring it's computationally infeasible to find two distinct inputs that produce the same hash value. Additionally, **Pre-image Resistance** is essential, where given a hash value, it should be impractical to retrieve the original input. Furthermore, Second Pre-image Resistance is crucial, meaning it's difficult to find another input that produces the same hash as a given input. Hash functions should exhibit the **Avalanche Effect**, where a slight change in the input generates a significantly different hash output. They should also be **Deterministic**, meaning the same input will always yield the same hash value. Overall, these properties collectively fortify the integrity and security of hash functions, making them invaluable for numerous cryptographic and security applications.
4. The proposed method of dividing the message into 128-bit chunks, XORing them together, and then applying a standard message digest like MD5 or SHA1 on the result does not create a secure hash

function. While this process might be faster than traditional hashing algorithms, it lacks crucial properties necessary for a robust hash function.

XORing the message chunks together effectively creates a compression function, but it significantly weakens the resulting hash's security. XOR operations do not offer the necessary avalanche effect, where small changes in the input significantly alter the output. This method also doesn't provide collision resistance or pre-image resistance, critical aspects of a secure hash function.

Additionally, using MD5 or SHA1, which are already deemed weak due to vulnerabilities discovered over time, further compromises the overall security. Both MD5 and SHA1 have known collision vulnerabilities, making them unsuitable for modern cryptographic applications where security is paramount.

In summary, while the proposed method might offer increased speed, it sacrifices essential security properties, making it inadequate for ensuring message integrity and security. Modern hash functions should prioritize collision resistance, pre-image resistance, and other critical security properties, which this method lacks.

5. In the Diffie-Hellman key exchange:

Given:

$p = 7$ and $g = 3$

- Alice sends $A = 6$ to Bob

- Bob sends $B = 5$ to Alice

The private keys of Alice (S_A) and Bob (S_B) can be calculated using the formula:
Shared Secret Key = $g^{S_A * S_B} \mod p$

Calculating Alice's Private Key (S_A):

So,

$$S_A = 3^5 \mod 7$$

Thus, $S_A = 5$.

Calculating Bob's Private Key (S_B):

So,

$$S_B = 3^6 \mod 7$$

Thus, $S_B = 1$.

Calculating the Shared Secret Key:

The shared secret key that Alice and Bob agree upon is given by:

$$1^5 \mod 7 = 1$$

Therefore, Alice's private key S_A is 5, Bob's private key S_B is 1, and the shared secret key on which they agree is 1.

6. A. Encryption using Public Key:

$$C = M^e \mod n$$

B. Decryption using Private Key:

$$M = C^d \mod n$$

C. Proof:

$$(1) P \equiv P^{ed} \pmod{p}$$

$$(2) P \equiv P^{ed} \pmod{q}.$$

Because p and q are relative primes. Now since:

$$e * d \equiv 1 \pmod{(p-1)(q-1)}$$

We have:

$$e * d - 1 = (q-1)(p-1)k$$

So, if q does not divide P , we have in (2):

$$P \equiv (P^{(q-1)(p-1)k} * P) \pmod{q}$$

Now by Fermat's theorem we can conclude:

$$P^{ed} \equiv (P^{(q-1)(p-1)k} * P) \equiv 1^{(p-1)k} * P \equiv P \pmod{q}.$$

D. In RSA, signing a plaintext M using the private key involves taking the hash of the message and the encrypting this hash value with the private key d to generate the signature S . The formula for generating the signature is:

$$S = M^d \pmod{n}$$

E. To verify that the received message M and its associated signature S were generated by Alice, Bob needs Alice's public key. To verify the signature:

1. Bob retrieves Alice's public key e and n .
2. Bob decrypts the received signature S using Alice's public key e :
 - If $M = S^e \pmod{n}$, then the signature is valid.
 - If the decrypted value matches the original message M , the signature is genuine and was indeed generated by Alice using her private key.

This process confirms both the authenticity of the message and the fact that the signature was produced by someone possessing the corresponding private key (in this case, Alice).